SETUP GUIDE

DOCKER

AI Platform Engineering (Module 5)

This guide covers installation and verification of:

1. Docker Engine / Docker Desktop
2. Docker Compose
3. NVIDIA Container Toolkit (optional — only if you have an NVIDIA GPU)
4. Building and running a test ML container

IMPORTANT: Docker is used in Weeks 4–11. Complete this setup fully.

PREREQUISITES

Before starting, ensure you have:

- [ ] 20+ GB free disk space (ML images are large)
- [ ] 8+ GB RAM total system memory (4 GB allocated to Docker)
- [ ] Admin/sudo access for installation
- [ ] Internet connection for pulling images

For GPU support (optional):

- [ ] NVIDIA GPU (check: does your machine have one?)
- [ ] NVIDIA GPU driver installed (nvidia-smi should work on host)

SECTION A: DOCKER INSTALLATION

========== Windows (via WSL 2) ==========

Docker Desktop for Windows uses WSL 2 as its backend.

Step 1: Ensure WSL 2 is enabled

```
# In PowerShell (admin):
wsl --status
# Should show: Default Version: 2

# If not installed:
wsl --install
```

```
# Restart computer, then:
wsl --set-default-version 2
```

Step 2: Install Docker Desktop

1. Download Docker Desktop from: https://www.docker.com/products/docker-desktop/
2. Run the installer
3. During setup, ensure "Use WSL 2 instead of Hyper-V" is checked
4. Restart computer after installation
5. Launch Docker Desktop from Start menu
6. Wait for "Docker Desktop is running" in system tray

Step 3: Configure Docker Desktop

1. Open Docker Desktop → Settings (gear icon)
2. General:
   Use the WSL 2 based engine
   Start Docker Desktop when you log in (optional)
3. Resources → WSL Integration:
   Enable integration with your Ubuntu distro
4. Resources → Advanced:
   Memory: 4 GB minimum (8 GB recommended for ML)
   Disk image size: 60+ GB

NOTE: If you use the WSL 2 backend, the memory/CPU sliders will NOT appear. Instead, WSL 2 automatically shares your system RAM (up to 50%).

This is fine — no action needed. If you ever need to limit it, create C:\Users\<YourName>\.wslconfig with [wsl2] memory=8GB.

5. Docker Engine (JSON config):
   Add: "features": {"buildkit": true}
6. Click "Apply & Restart"

Step 4: Verify in WSL terminal

```
# Open WSL (Ubuntu) terminal:
docker --version

# Expected: Docker version 24.x.x or higher

docker compose version

# Expected: Docker Compose version v2.x.x or higher
```

docker run hello-world

Expected: "Hello from Docker!" message

NOTE: If the pull fails with "EOF" or timeout on the first attempt,

simply re-run the same command. Docker image pulls sometimes fail on

the first try due to CDN/network hiccups — this is normal.

========== macOS ==========

Step 1: Install Docker Desktop

Option A: Download from website
https://www.docker.com/products/docker-desktop/
Choose Apple Silicon (M1/M2/M3) or Intel chip version

Option B: Homebrew
brew install --cask docker
Launch Docker Desktop from Applications
Wait for whale icon in menu bar to show "running"

Step 2: Configure

1. Docker Desktop → Settings → Resources:
   Memory: 4 GB minimum (8 GB recommended)
   Disk image size: 60+ GB
2. Docker Engine:
   Add: "features": {"buildkit": true}
3. Apply & Restart

Step 3: Verify

docker --version
docker compose version
docker run hello-world

========== Linux (Ubuntu/Debian) ==========

Step 1: Remove old versions

sudo apt-get remove docker docker-engine docker.io containerd runc 2>/dev/null

Step 2: Install Docker Engine

Add Docker's official GPG key

sudo apt-get update

```
sudo apt-get install -y ca-certificates curl gnupg

sudo install -m 0755 -d /etc/apt/keyrings

curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg

sudo chmod a+r /etc/apt/keyrings/docker.gpg

# Add the repository

echo \

"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu \

$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \

      sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Install Docker

sudo apt-get update

sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-
plugin
```

Step 3: Post-installation (run without sudo)

```
sudo groupadd docker 2>/dev/null

sudo usermod -aG docker $USER

newgrp docker

# Verify no-sudo access:

docker run hello-world
```

Step 4: Enable on boot

```
sudo systemctl enable docker.service

sudo systemctl enable containerd.service
```

Step 5: Configure Docker daemon

```
sudo tee /etc/docker/daemon.json << 'EOF'

{

"log-driver": "json-file",
```

"log-opts": {

"max-size": "10m",

"max-file": "3"

},

"features": {

"buildkit": true

}

}

EOF

sudo systemctl restart docker

Step 6: Verify

docker --version
docker compose version
docker run hello-world

SECTION 2: NVIDIA CONTAINER TOOLKIT (OPTIONAL)

Skip this section if you do NOT have an NVIDIA GPU.

Step 1: Verify GPU driver on host

nvidia-smi

Expected: Shows GPU name, driver version, CUDA version

Note the driver version (e.g., 535.xx) — this determines max CUDA in containers

Step 2: Install NVIDIA Container Toolkit

Ubuntu/Debian/WSL:

curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit-keyring.gpg

curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-toolkit.list | \

sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg] https://#g' | \

sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list

sudo apt-get update

sudo apt-get install -y nvidia-container-toolkit

Step 3: Configure Docker to use NVIDIA runtime

sudo nvidia-ctk runtime configure --runtime=docker

sudo systemctl restart docker

Step 4: Verify GPU access in containers

docker run --rm --gpus all nvidia/cuda:12.1.0-base-ubuntu22.04 nvidia-smi

Expected: Same GPU info as host nvidia-smi, running INSIDE the container

If this works → GPU containers are ready!

--- Windows (Docker Desktop) Note ---

Docker Desktop with WSL 2 backend supports GPU passthrough automatically

if you have:

- NVIDIA GPU driver for Windows installed
- WSL 2 with GPU support (Windows 11 or Windows 10 21H2+)
- Docker Desktop configured to use WSL 2 engine

No separate NVIDIA Container Toolkit install needed on Windows — Docker

Desktop handles it. Verify with:

```
docker run --rm --gpus all nvidia/cuda:12.1.0-base-ubuntu22.04 nvidia-smi
```

SECTION 3: BUILD AND RUN TEST ML CONTAINER

This exercise verifies your full Docker setup end-to-end.

Step 1: Create project directory

mkdir -p ~/docker-ml-test && cd ~/docker-ml-test

Step 2: Create requirements.txt

cat << 'EOF' > requirements.txt

```
scikit-learn==1.3.2

pandas==2.1.4

numpy==1.26.2

EOF
```

Step 3: Create training script

```
cat << 'EOF' > train.py

import pandas as pd

import numpy as np

from sklearn.linear_model import LogisticRegression

from sklearn.datasets import make_classification

from sklearn.model_selection import train_test_split

import json

import os

print("=" * 50)

print("ML Training Container - Week 4 Test")

print("=" * 50)

# Generate sample data

print("\n[1/4] Generating dataset...")

X, y = make_classification(n_samples=1000, n_features=10, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"  Train samples: {len(X_train)}, Test samples: {len(X_test)}")

# Train model

print("\n[2/4] Training model...")

model = LogisticRegression(max_iter=200)

model.fit(X_train, y_train)

# Evaluate
```

```
print("\n[3/4] Evaluating...")

train_acc = model.score(X_train, y_train)

test_acc = model.score(X_test, y_test)

print(f"  Train accuracy: {train_acc:.4f}")

print(f"  Test accuracy:  {test_acc:.4f}")

# Save metrics

print("\n[4/4] Saving metrics...")

metrics = {

"train_accuracy": round(train_acc, 4),

"test_accuracy": round(test_acc, 4),

"n_train": len(X_train),

"n_test": len(X_test)

}

os.makedirs("/app/output", exist_ok=True)

with open("/app/output/metrics.json", "w") as f:

json.dump(metrics, f, indent=2)

print(f"  Metrics saved to /app/output/metrics.json")

print("\n" + "=" * 50)

print("SUCCESS: Training complete!")

print("=" * 50)

EOF
```

Step 4: Create Dockerfile

```
cat << 'EOF' > Dockerfile

FROM python:3.10-slim

WORKDIR /app
```

```
# Install dependencies (cached layer)

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

# Copy application code (changes frequently)

COPY train.py .

# Create output directory

RUN mkdir -p /app/output

CMD ["python", "train.py"]

EOF
```

Step 5: Create .dockerignore

```
cat << 'EOF' > .dockerignore

__pycache__/

*.pyc

.git/

.env

venv/

EOF
```

Step 6: Build the image

```
docker build -t ml-test:v1 .

# Expected: Successfully built, tagged ml-test:v1

# First build takes 1-2 minutes (downloading base image + packages)
```

Step 7: Run the container

```
docker run --rm ml-test:v1

# Expected: Training output with accuracy metrics and "SUCCESS" message
```

Step 8: Run with volume (persist output)

```
docker run --rm -v $(pwd)/output:/app/output ml-test:v1

# Check output on host:
```

```
cat output/metrics.json

# Expected: JSON with train_accuracy, test_accuracy
```

Step 9: Verify layer caching

```
# Make a small code change:

echo "# test change" >> train.py

# Rebuild:

docker build -t ml-test:v2 .

# Expected: pip install layer uses cache ("CACHED"), only code layer rebuilds

# Build should complete in seconds, not minutes
```

Step 10: Test networking (two containers)

```
# Create a network

docker network create ml-net

# Run a simple web server

docker run -d --network ml-net --name web-test python:3.10-slim \

    python -m http.server 8080

# Test connectivity from another container

docker run --rm --network ml-net python:3.10-slim \

    python -c "import urllib.request; print(urllib.request.urlopen('http://web-test:8080').status)"

# Expected: 200

# Cleanup
docker stop web-test && docker rm web-test
docker network rm ml-net
```

SECTION 4: DOCKER COMPOSE TEST

Step 1: Create compose project

```
mkdir -p ~/compose-ml-test && cd ~/compose-ml-test
```

Step 2: Create docker-compose.yml

```
cat << 'EOF' > docker-compose.yml
services:
mlflow:
image: ghcr.io/mlflow/mlflow:v2.9.2
ports:
"5000:5000"
command: mlflow server --host 0.0.0.0 --port 5000
volumes:
mlflow-data:/mlflow
training:
build: ../docker-ml-test
environment:
MLFLOW_TRACKING_URI=http://mlflow:5000
depends_on:
mlflow
volumes:
mlflow-data:

EOF
```

Step 3: Start the stack

```
docker compose up -d
# Expected: Both services start

docker compose ps
# Expected: Both services "running"

# Check MLflow UI: open http://localhost:5000 in browser
```

Step 4: View logs

```
docker compose logs training
# Expected: Training output

docker compose logs mlflow
# Expected: MLflow server startup logs
```

Step 5: Cleanup

```
docker compose down -v
# Stops services, removes containers, removes volumes
```

SECTION 5: TROUBLESHOOTING

--- Docker Installation Issues ---

Problem: "docker: command not found" (Linux)
Fix: Ensure Docker was installed:
sudo apt-get install docker-ce docker-ce-cli containerd.io

Then: sudo systemctl start docker

Problem: "permission denied" when running docker commands (Linux)
Fix: Add user to docker group:
sudo usermod -aG docker $USER

Then log out and log back in (or: newgrp docker)

Problem: Docker Desktop won't start (Windows)
Fix: Ensure WSL 2 is installed: wsl --install
Ensure virtualization is enabled in BIOS
Restart computer after Docker Desktop install

Problem: Docker Desktop won't start (macOS)
Fix: Check macOS version (requires 12.0+)
Try: rm -rf ~/Library/Group\ Containers/group.com.docker
Reinstall Docker Desktop

--- Build Issues ---

Problem: "no space left on device"
Fix: Clean up unused images and containers:
docker system prune -a

docker volume prune

(Warning: removes all unused images and volumes)

Problem: Build fails at pip install (network timeout)
Fix: Retry (transient network issues are common)
Or add: RUN pip install --timeout 120 -r requirements.txt

Problem: Build fails with "exec format error"
Fix: Architecture mismatch (e.g., building ARM image, running on x86)
Specify platform: docker build --platform linux/amd64 .

--- Runtime Issues ---

Problem: Container exits immediately
Fix: Check logs: docker logs <container-id>
Common causes: missing file, import error, wrong CMD

Problem: "port already in use"
Fix: Another service is using that port
Find it: lsof -i :5000 (or netstat -tlnp | grep 5000)
Kill it: kill <PID>
Or use a different host port: -p 5001:5000

Problem: Container cannot access GPU
Fix: Verify host driver: nvidia-smi
Verify toolkit: docker run --gpus all nvidia/cuda:12.1.0-base-ubuntu22.04 nvidia-smi
If toolkit not installed: follow Section 2 above
Check CUDA compatibility: container CUDA must not exceed host driver support

Problem: Volume permission denied
Fix: Container runs as non-root but volume is owned by root
Add to Dockerfile: RUN chown -R appuser:appuser /app/output
Or run container with matching UID: docker run --user $(id -u):$(id -g)

--- Docker Compose Issues ---

Problem: "service not found" or YAML parse error
Fix: Check indentation (YAML is space-sensitive)
Validate: docker compose config

Problem: Service cannot reach another service
Fix: Ensure both services are in the same network (default Compose network)
Use service name as hostname: http://mlflow:5000 not localhost:5000
Check depends_on for startup order

Problem: "image not found" for local build
Fix: Use "build: ./path" instead of "image: name" for local Dockerfiles
Run: docker compose build before docker compose up

SECTION 6: VERIFICATION CHECKLIST

Complete these checks before the live session:

Docker Core (Required):

- [ ] docker --version shows 24.x+ (or 25.x+)
- [ ] docker compose version shows v2.x+
- [ ] docker run hello-world succeeds
- [ ] Docker has 4+ GB RAM allocated
- [ ] docker info shows buildkit enabled

Build & Run (Required):

- [ ] Built ml-test:v1 image successfully
- [ ] Container runs and shows training output with metrics
- [ ] Volume mount works (metrics.json appears on host)
- [ ] Layer caching works (second build is faster)

Networking (Required):

- [ ] Custom network created successfully
- [ ] Container-to-container communication works by name

Docker Compose (Recommended):

- [ ] docker compose up starts multiple services
- [ ] Services communicate (training finds MLflow)
- [ ] docker compose down cleans up

GPU Support (Optional):

- [ ] nvidia-smi works on host
- [ ] docker run --gpus all nvidia/cuda:12.1.0-base-ubuntu22.04 nvidia-smi works
- [ ] GPU visible inside container

Cleanup commands:
Remove test containers and images:

```
docker rm -f $(docker ps -aq) 2>/dev/null

docker rmi ml-test:v1 ml-test:v2 2>/dev/null

docker system prune -f
```

Keep Docker installed — it's used through Week 11!

SECTION 7: USEFUL DAILY COMMANDS REFERENCE

```
# Build

docker build -t name:tag .

docker build --no-cache -t name:tag .

# Run

docker run --rm name:tag                    # Run and auto-remove

docker run -d -p 8080:8080 name:tag         # Detached with port

docker run -v $(pwd)/data:/app/data name    # With volume

docker run --gpus all name:tag              # With GPU

# Inspect

docker ps                                   # Running containers

docker logs <id>                            # Container output

docker exec -it <id> bash                   # Shell into container

docker stats                                # Live resource usage

# Cleanup

docker stop $(docker ps -q)                 # Stop all running

docker sy stem prune -a                     # Remove all unused

docker volume prune                         # Remove unused volumes

# Compose

docker compose up -d                        # Start stack

docker compose down                         # Stop stack

docker compose logs -f                      # Follow all logs

docker compose exec <service> bash          # Shell into service
```